CS-3710: Operating Systems

Assignment 1

Process Management

Detailed Requirements and Specifications

Submission Deadline: October 16, EoD

Group Size: 2–4 students

Submission Components:

1. Complete source code with proper documentation and README
2. Demonstration-cum-viva

Implementation Language: C is the preferred language. Other languages are permitted; additional credit may be awarded for implementations written in C.

Assignment Scope: Process Management only.
Memory Management and Concurrency are intentionally outside the scope of Assignment 1 and will be introduced in later submissions.

1 Assignment Overview

Assignment 1 is the first stage of the Operating Systems scenario-based project.

Your group must choose **one** of the project scenarios described in the accompanying *Project Overview* document:

1. Smart Salon Management
2. Smart Library Management
3. Smart Restaurant Management
4. Emergency Hospital Management

The scenario you choose will remain your group's project throughout the subsequent submissions.

The purpose of Assignment 1 is to model the entities in your selected real-world scenario as **processes** and build a working simulation of their scheduling and execution.

The project overview describes the overall progression from a real-world resource-management problem to an Operating Systems model. Assignment 1 focuses exclusively on the first part of that progression:

Process Management

The remaining components of the overall project will be introduced in later submissions.

2 Learning Objectives

By completing this assignment, you should be able to:

- Model real-world entities as Operating Systems processes.

- Represent process arrival, execution, completion, and waiting.
- Implement and compare multiple CPU-style scheduling algorithms.
- Understand the difference between preemptive and non-preemptive scheduling where applicable.
- Calculate and interpret standard process-scheduling metrics.
- Design a deterministic discrete-event simulation.
- Reason about the trade-offs between different scheduling policies.
- Explain and defend implementation and design decisions.

3 Scope of Assignment 1

What is included in A1?

Assignment 1 is restricted to **Process Management**.
The system must model:

- Processes/entities arriving over time.
- A ready queue or equivalent scheduling structure.
- One or more identical service units.
- Process scheduling according to the required scheduling algorithms.
- Process execution and completion.
- Preemption where explicitly required.
- Scheduling metrics.
- Comparison of the required scheduling algorithms.

For the purposes of this assignment, the scenario's primary service providers are treated as the available parallel service units.

Intentionally Out of Scope
The following are **not required for Assignment 1**:

- First Fit, Best Fit, or Worst Fit memory allocation.
- Memory fragmentation.
- Locks, mutexes, semaphores, or synchronization primitives.
- Race conditions.
- Deadlock detection or prevention.
- Concurrent access to shared resources.
- Integration of Process Management with Memory Management and Concurrency.
- The "Final Challenge" described in the Project Overview.

These topics belong to later stages of the project.

4 Scenario-Specific Requirements

Although the four projects use different real-world settings, Assignment 1 follows the same underlying process-management structure.

The following table defines the scenario-specific mapping for A1.

Project	**Process**	**Primary Service Units**	**Required Scheduling Algorithms**
Smart Salon	Customer	2 barbers	FCFS; Priority Scheduling; Round Robin; Preemptive and non-preemptive Priority Scheduling
Smart Library	Reader	2 librarians	FCFS; Priority Scheduling
Smart Restaurant	Order	2 chefs	FCFS; Priority Scheduling; Shortest Job First (SJF)
Emergency Hospital	Patient	2 doctors	FCFS; Priority Scheduling; Preemptive and non-preemptive Priority Scheduling

The scheduling algorithms listed above are based on the scheduling concepts associated with each scenario in the Project Overview.

Important: You are required to implement **all** scheduling algorithms listed for your selected project. Implementing only one or two of them is not sufficient for a complete submission.

5 Process Model

Each entity in the selected scenario should be represented by a process.

At minimum, every process must contain the following information:

`Field`	**Description**
`ID`	Unique identifier for the process.
`Arrival Time`	The time at which the process enters the system and becomes eligible for scheduling.
`Priority`	Scheduling priority. A lower numerical value represents a higher priority.
`Service Time`	Total amount of execution time required by the process.
`Category / Type`	A scenario-specific label such as VIP, Regular, Faculty, Researcher, etc. This is primarily descriptive unless otherwise specified.

The terminology used for "service time" depends on the selected scenario:

- **Salon:** duration of the haircut/service.
- **Library:** simulated duration of the reader's service interaction.
- **Restaurant:** preparation time required for the order.
- **Hospital:** simulated treatment time requiring a doctor.

The simulation should treat service time as the process's equivalent of a CPU burst.

6 Process Lifecycle

Your implementation should model, either explicitly or implicitly, the following process lifecycle:

$$\texttt{NEW} \rightarrow \texttt{READY} \rightarrow \texttt{RUNNING} \rightarrow \texttt{TERMINATED}$$

A process may move from:

- **NEW** to **READY** when it arrives.
- **READY** to **RUNNING** when selected by the scheduler.
- **RUNNING** to **TERMINATED** when its service time is completed.
- **RUNNING** back to **READY** if it is preempted.

The implementation need not use a particular data structure or class hierarchy for these states. However, the behaviour of the system must correspond to this model.

7 Required Scheduling Algorithms

You must implement every scheduling algorithm listed for your selected scenario in Section 4.

Unless otherwise stated, all algorithms operate on processes that have already arrived and are waiting to be scheduled.

7.1 First-Come, First-Served (FCFS)

FCFS schedules processes in order of arrival.

Rules:

- The process with the earliest arrival time is selected first.
- FCFS is **non-preemptive**.
- Once a process begins execution, it continues until completion.
- If multiple processes arrive at the same time, the lower process ID in lexicographic order is selected first.

7.2 Priority Scheduling

Priority Scheduling selects the ready process with the highest priority.

For this assignment:

Smaller numerical priority value = Higher priority

For example:

$$\text{Priority 1} > \text{Priority 2} > \text{Priority 3}$$

Unless the scenario specifically requires the preemptive variant, Priority Scheduling is **non-preemptive**.

Tie-breaking order:

1. Higher priority
2. Earlier arrival time
3. Lexicographically smaller process ID

7.3 Preemptive Priority Scheduling

For projects where preemptive scheduling is required, the system must also implement the preemptive version of Priority Scheduling.

A running process may be interrupted when a newly available process has a strictly higher priority.

Rules:

- A process is preempted only when another ready process has a strictly higher priority.
- Equal-priority processes do not cause preemption.
- A preempted process returns to the READY state with its remaining service time preserved.
- Scheduling must be reevaluated whenever a relevant process arrives or a running process completes.

For systems with multiple service units, preemption should occur when a ready process has higher priority than at least one currently running process.

When choosing which running process to preempt:

1. Preempt the running process with the lowest priority.
2. If priorities are equal, preempt the process with the later arrival time.
3. If these are also equal, use lexicographically larger process ID.

This rule ensures deterministic behaviour.

7.4 Round Robin

Round Robin is required for the Smart Salon project.

Rules:

- A fixed time quantum is used.
- When a process exhausts its quantum but has remaining service time, it is preempted and placed at the back of the ready queue.
- When a process completes before the quantum expires, it is removed from the system.
- Newly arrived processes are inserted into the ready queue in arrival order.
- If a process's quantum expires at the same time as new processes arrive, the newly arrived processes are added before the preempted process is reinserted at the back of the queue.

The time quantum is provided as part of the standardized input described later in this document.

7.5 Shortest Job First (SJF)

SJF is required for the Smart Restaurant project.

SJF selects the ready process with the shortest total service time.

Rules:

- SJF is **non-preemptive** for Assignment 1.
- A process that has already started execution continues until completion.
- The shortest available service time is selected at each scheduling decision.

Tie-breaking order:

1. Shorter service time
2. Earlier arrival time
3. Lexicographically smaller process ID

8 Multiple Service Units

The scenarios contain two primary service units:

Scenario	Number of Service Units
Smart Salon	2 barbers
Smart Library	2 librarians
Smart Restaurant	2 chefs
Emergency Hospital	2 doctors

For Assignment 1, these service units are treated as **two identical parallel execution slots**.
The scheduler therefore may have up to two processes executing simultaneously.

You do not need to model the internal differences between the individual barbers, librarians, chefs, or doctors. All service units are equivalent for A1 scheduling purposes.

When multiple processes are eligible to start at the same time, the scheduler should fill available service slots according to the selected scheduling policy.

9 Simulation Model

The program should behave as a discrete-event simulation.
At minimum, the simulation must be capable of representing the following events:

- Process arrival
- Process starting execution
- Process preemption, where applicable
- Process completion

The simulation clock should advance based on relevant event times rather than depending on real-world wall-clock time.

Your implementation should therefore produce the same result every time the same input is given to the same scheduling algorithm.

> This is a **simulation**. You are not required to create real-world delays such as making the program actually wait ten seconds to represent ten minutes of service.
> A service time of 10 simply means that the process requires 10 simulation time units of execution.

10 Required Scheduling Metrics

Your system must calculate the following metrics for every process.
Let:

- A_i = arrival time of process i
- S_i = first start time of process i
- C_i = completion time of process i
- B_i = total required service time of process i

10.1 Completion Time

$$C_i = \text{time at which process } i \text{ finishes execution}$$

10.2 Turnaround Time

$$TAT_i = C_i - A_i$$

10.3 Response Time

$$RT_i = S_i - A_i$$

Response time is based on the **first time** the process receives service, even if it is later preempted.

10.4 Waiting Time

$$WT_i = TAT_i - B_i$$

For preemptive and Round Robin scheduling, waiting time therefore includes all periods during which the process is ready but not executing.

10.5 Throughput

For the overall simulation:

$$Throughput = \frac{N}{\text{last completion time} - \text{first arrival time}}$$

where N is the total number of completed processes.
Throughput should be reported in processes per simulation time unit.

10.6 Service-Unit Utilization

For Assignment 1, "resource utilization" refers only to utilization of the primary service units used for process execution.

Let:

- M = number of service units
- B = total busy time summed across all service units
- E = simulation elapsed time

Then:

$$Utilization = \frac{B}{M \times E} \times 100$$

For A1, $M = 2$ for all four project scenarios.

The utilization of the barbers, librarians, chefs, or doctors should be reported as a percentage.

11 Algorithm Comparison

A central purpose of this assignment is not merely to implement scheduling algorithms, but to **observe how they behave differently**.

Your documentation must therefore include a comparison of all required algorithms for your selected project.

At minimum, discuss:

- Average waiting time

- Average turnaround time
- Average response time
- Total / average completion behaviour
- Throughput
- Service-unit utilization
- Situations in which one scheduling policy behaves differently from another
- Any starvation or fairness concerns that arise

The comparison should be based on actual simulation results rather than only theoretical statements.

You may use tables, graphs, Gantt charts, or other visualizations in your README or demonstration.

12 Standardized Input Specification

To allow the teaching staff to run standardized tests against your submitted implementations, all groups must follow the input format below.

Each project uses the same structural format, with scenario-specific names for the process entity and category field.

12.1 File Format

The input file must contain:

1. Number of processes
2. Round Robin time quantum
3. One process record per line

The format is:

```
N
Q
ID,ARRIVAL,PRIORITY,SERVICE,CATEGORY
ID,ARRIVAL,PRIORITY,SERVICE,CATEGORY
...
```

where:

- N is the number of processes.
- Q is the Round Robin time quantum.
- ID is a unique process identifier.
- $ARRIVAL$ is the process arrival time.
- $PRIORITY$ is the scheduling priority, where a smaller number means a higher priority.
- $SERVICE$ is the total service time required by the process.
- $CATEGORY$ is a scenario-specific label.

12.2 Input Constraints

Unless explicitly stated otherwise:

- $N \geq 1$.
- Arrival times are non-negative integers.
- Service times are positive integers.
- Priorities are positive integers.
- The Round Robin quantum is a positive integer.
- Process IDs are unique within an input file.
- Category values contain no commas.
- Processes may arrive at the same time.
- Multiple processes may have the same priority.

12.3 Scenario-Specific Input Records

Project	**Process ID**	**SERVICE represents**	**CATEGORY examples**
Smart Salon	Customer ID	Haircut/service duration	REGULAR, VIP
Smart Library	Reader ID	Simulated service duration	REGULAR, FACULTY, RESEARCHER
Smart Restaurant	Order ID	Preparation time	NORMAL, VIP
Emergency Hospital	Patient ID	Treatment duration	Condition/type such as FEVER, FRACTURE, SEVERE

The `CATEGORY` field is included so that the input remains faithful to the real-world scenario. Scheduling algorithms must use the explicitly provided numerical `PRIORITY` value rather than attempting to infer priority from the category string.

13 Sample Inputs

The following examples illustrate the standardized format.

13.1 Smart Salon

```
5
2
C1,0,3,8,REGULAR
C2,1,1,4,VIP
C3,2,3,5,REGULAR
C4,3,2,6,REGULAR
C5,4,1,3,VIP
```

13.2 Smart Library

```
5
2
R1,0,3,7,REGULAR
R2,1,1,4,FACULTY
R3,2,3,5,REGULAR
R4,3,2,6,RESEARCHER
R5,4,1,3,FACULTY
```

13.3 Smart Restaurant

```
5
2
O1,0,2,8,NORMAL
O2,1,1,4,VIP
O3,2,2,5,NORMAL
O4,3,1,6,VIP
O5,4,2,3,NORMAL
```

13.4 Emergency Hospital

```
5
2
P1,0,3,10,FEVER
P2,1,2,8,FRACTURE
P3,2,1,12,CRITICAL
P4,3,3,5,MINOR
P5,4,1,7,SEVERE
```

These are illustrative examples only. You should create additional inputs to test different scheduling behaviours and edge cases.

14 Standardized Command-Line Interface

A **CLI implementation is required** for Assignment 1.

For interoperability with the teaching staff's automated tests, your program must support the following logical invocation:

```
program INPUT_FILE ALGORITHM
```

where `ALGORITHM` is one of:

```
FCFS
PRIORITY
PRIORITY_PREEMPTIVE
RR
SJF
```

Only algorithms applicable to the selected project need to be supported.

For example, a Smart Restaurant implementation is expected to support:

```
FCFS
PRIORITY
SJF
```

while a Smart Salon implementation is expected to support:

```
FCFS
PRIORITY
PRIORITY_PREEMPTIVE
RR
```

The exact executable name is up to you. The above describes the required interface between your program and the input/test harness.

Note: You may choose to implement a graphical user interface if you prefer. You do not need to implement the CLI functionality in that case. However, your UI should support the above funtionality for it to be considered.

15 Standardized Output

The normal presentation of your program is up to your group, provided the required information can be clearly demonstrated.

However, the CLI(/GUI) must produce a **machine-readable result** so that the teaching staff can test the implementation automatically.

The required output structure is:

```
PROCESS_ID,ARRIVAL,PRIORITY,SERVICE,START,COMPLETION,WAITING,TURNAROUND,RESPONSE
...
SUMMARY
AVERAGE_WAITING,...
AVERAGE_TURNAROUND,...
AVERAGE_RESPONSE,...
THROUGHPUT,...
UTILIZATION,...
```

There must be exactly one process-result row for every completed process.

The values in the process-result row correspond to:

- `START`: first time the process starts execution.
- `COMPLETION`: final completion time.
- `WAITING`: total waiting time.
- `TURNAROUND`: turnaround time.
- `RESPONSE`: response time.

The summary values must correspond to the metrics defined in Section 10.

Formatting requirements:

- Use decimal notation for non-integer summary metrics where needed.
- Do not include explanatory prose within the machine-readable output.
- Process result rows should be emitted in lexicographic process-ID order.
- The simulation itself must not depend on the formatting of output.

The teaching staff may use private test inputs that are not released to students. The purpose of the standardized input/output interface is to allow your implementation to be tested consistently without releasing the test suite itself.

16 Required Functionality

Every submission must satisfy the following requirements.

16.1 Process Creation and Arrival

The program must:

- Read process information from the standardized input.
- Create a process representation for every input record.
- Place each process into the READY set at its specified arrival time.

16.2 Scheduling

The program must:

- Implement every scheduling algorithm required for the selected scenario.
- Follow the scheduling semantics and tie-breaking rules specified in this document.
- Correctly handle multiple processes arriving at the same time.
- Correctly handle idle periods when no process is ready.

16.3 Execution

The simulation must:

- Track remaining service time.
- Mark the first time each process begins execution.
- Track completion time.
- Correctly handle preemption where required.
- Correctly handle processes that complete before a time quantum expires.

16.4 Metrics

The program must calculate every metric specified in Section 10.

16.5 Algorithm Comparison

The documentation must compare the required algorithms using actual simulation results.

17 Edge Cases

Your implementation should correctly handle cases including:

- Only one process.
- All processes arriving at the same time.
- Processes arriving while another process is running.
- Very short processes.
- Very long processes.

- Equal priorities.
- Equal service times.
- Processes with identical arrival times, priorities, and service times.
- Long idle periods before the next process arrives.
- A process whose service time is exactly equal to the Round Robin quantum.
- A process whose service time is shorter than the Round Robin quantum.
- A process that requires multiple Round Robin quanta.
- Preemption immediately after the arrival of a higher-priority process, where applicable.

Groups are strongly encouraged to create their own test inputs for these cases.

18 Optional Advanced Features

Groups that wish to explore the Process Management component further may implement additional features.

Any optional feature must remain within the **Process Management** scope.

Examples include:

- Aging to reduce starvation in Priority Scheduling.
- A visual Gantt chart or scheduling timeline.
- Interactive comparison of scheduling algorithms.
- Context-switch counting or context-switch overhead modelling.
- Additional process states or richer process lifecycle modelling.
- Multi-level feedback queue scheduling.
- Preemptive variants of additional algorithms.
- Dynamic process-arrival generation.
- Visualization of queue lengths and process behaviour over time.

Optional features should **not** implement functionality belonging to Memory Management or Concurrency. For example, implementing locks, deadlock prevention, or First Fit memory allocation does not count as an Assignment 1 process-management extension.

19 Testing Expectations

A formal testing report or formal test suite is **not required** for Assignment 1.

However, your group is strongly encouraged to include basic tests in the repository to verify the correctness of the implementation.

You may include:

- Sample input files.
- Expected results.
- Scripts for running multiple cases.
- Unit tests for core data structures or scheduler components.
- Regression tests for bugs discovered during development.

The teaching staff may use their own private test suite in addition to the tests included by your group.

20 Code and Documentation Requirements

Your code should be:

- Properly structured and readable.
- Modular rather than implemented entirely in one large function.
- Appropriately commented where the logic is non-obvious.
- Free of unnecessary duplicated code.
- Accompanied by a README with clear setup and execution instructions.

Your README must include, at minimum:

1. Names of all group members.
2. The selected project scenario.
3. How to compile and run the program.
4. The required command-line arguments.
5. A description of your process model.
6. A description of each implemented scheduling algorithm.
7. Design decisions and assumptions.
8. Definitions of the metrics used.
9. Algorithm comparison and observations.
10. Any known limitations.
11. Any optional features implemented.

21 Demonstration-cum-Viva

The total duration of the demonstration-cum-viva will be approximately **10–15 minutes** per group. The session will contain both:

1. **Demonstration and explanation of the system**
2. **Questioning / viva**

During the demonstration, your group should be prepared to show:

- Your selected scenario.
- The process model used.
- Execution of the required scheduling algorithms.
- The scheduling results and calculated metrics.
- Differences observed between scheduling algorithms.
- Any optional features implemented.

During the viva, you may be asked about:

- Your implementation.
- Data structures and architecture.
- Scheduling algorithm behaviour.
- Design decisions and assumptions.
- Edge cases.
- Process states and lifecycle.
- The calculated metrics.
- Why your observed results differ between scheduling algorithms.
- Specific parts of your codebase.

Every group member is expected to understand the complete system.

Each group member should be able to explain the implementation, the design decisions, the scheduling algorithms, and the relevant Operating Systems concepts used in the project.

You may divide the system into parts to work on it, but each individual should understand and be able to explain the work done by the rest of the group.

A live modification of the code will **not** be required.

22 Evaluation

There is no separate marks split for individual components of the submission.

The assignment will be evaluated primarily through the **demonstration-cum-viva** and the submitted codebase.

The evaluation will consider:

- **Correctness of implementation:** whether the required scheduling algorithms and metrics behave correctly.
- **Completeness:** whether all required functionality for the selected scenario has been implemented.
- **Understanding:** whether the group can explain the Operating Systems concepts underlying the implementation.
- **Design decisions:** whether the group can explain and defend the architecture, data structures, algorithms, and assumptions used.
- **Demonstration quality:** whether the system can be clearly demonstrated and its behaviour interpreted.
- **Code quality and documentation:** whether the codebase is organized, readable, documented, and accompanied by a useful README.

23 Use of AI Tools

You are free to use AI tools as you see fit while working on the assignment.

Use of AI tools will **not carry a direct penalty**.

However, the use of AI does not change what is expected from your group during the demonstration-cum-viva.

You are expected to:

- Understand how your implementation works.
- Understand the Operating Systems concepts used.
- Understand and justify your design decisions.
- Be able to explain the algorithms implemented.
- Be able to explain assumptions, edge cases, and limitations.
- Be able to answer questions about the codebase.

The purpose of the viva is to assess the group's understanding of the system it has submitted, regardless of which tools were used during development.

24 Submission Requirements

Each group must submit:

1. **Complete source code**
    - All source files.
    - Build / compilation instructions.
    - README and relevant documentation.
    - Any sample inputs or tests included by the group.
2. **Demonstration-cum-viva**
    - The working implementation must be demonstrated during the scheduled session.
    - All group members must be present and prepared to answer questions.

25 Submission Checklist

Before submitting, make sure that your group has completed the following:

- Implemented all scheduling algorithms required for that scenario.
- Correctly modelled process arrival and execution.
- Correctly handled multiple service units.
- Implemented preemption where required.
- Calculated completion time.
- Calculated waiting time.
- Calculated turnaround time.
- Calculated response time.

- Calculated throughput.
- Calculated service-unit utilization.
- Followed the standardized input format.
- Followed the standardized CLI interface.
- Produced the required machine-readable output.
- Documented and compared the scheduling algorithms.
- Included a complete README.
- Tested important edge cases.

Remember the project philosophy

"First solve the real-world problem, then discover the Operating System."

For Assignment 1, your focus is on discovering and implementing **Process Management** within your selected scenario.
The subsequent submissions will build on this system by introducing Memory Management, Concurrency, and finally the integration of all three components into the complete project.